
Diseño Arquitectónico - Parte 3

— INFO248 —

Dra. Valeria Henríquez



CQRS

¿Qué es?

Es el acrónimo de Command Query Responsibility Segregation. **CQRS es un patrón de arquitectura que separa los modelos para leer y escribir datos.** Para algunas situaciones, esta separación puede ser valiosa, pero se debe tener en cuenta que para la mayoría de los sistemas **CQRS agrega una complejidad arriesgada.**

<https://martinfowler.com/bliki/CQRS.html>



LA IDEA BÁSICA DE CQRS

La idea básica es que puedes dividir las operaciones de un sistema en dos categorías claramente diferenciadas (*CommandQuerySeparation*):

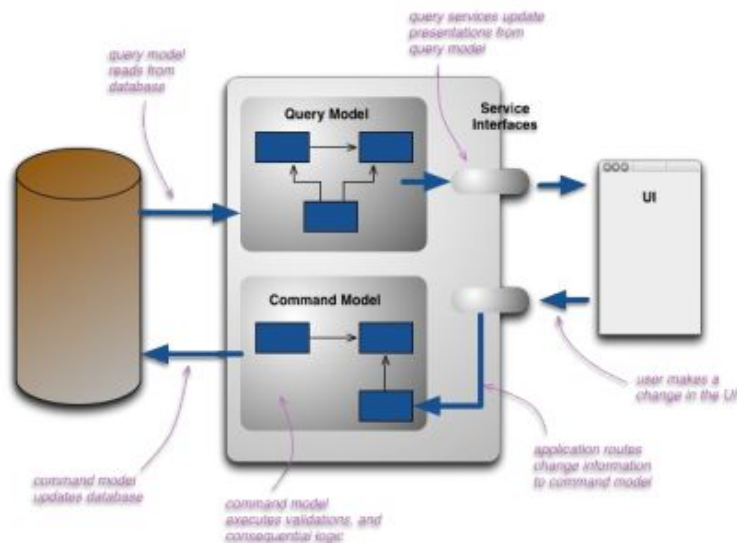
- **Consultas (Query Model).** Devuelven un resultado sin cambiar el estado del sistema y no tienen efectos secundarios.
- **Comandos (Command Model).** Cambian el estado de un sistema pero no devuelven un valor.

El cambio que **CQRS introduce** al enfoque dominante que la gente utiliza para interactuar con un sistema de información, **es dividir el modelo conceptual en modelos separados para actualizar y mostrar.**

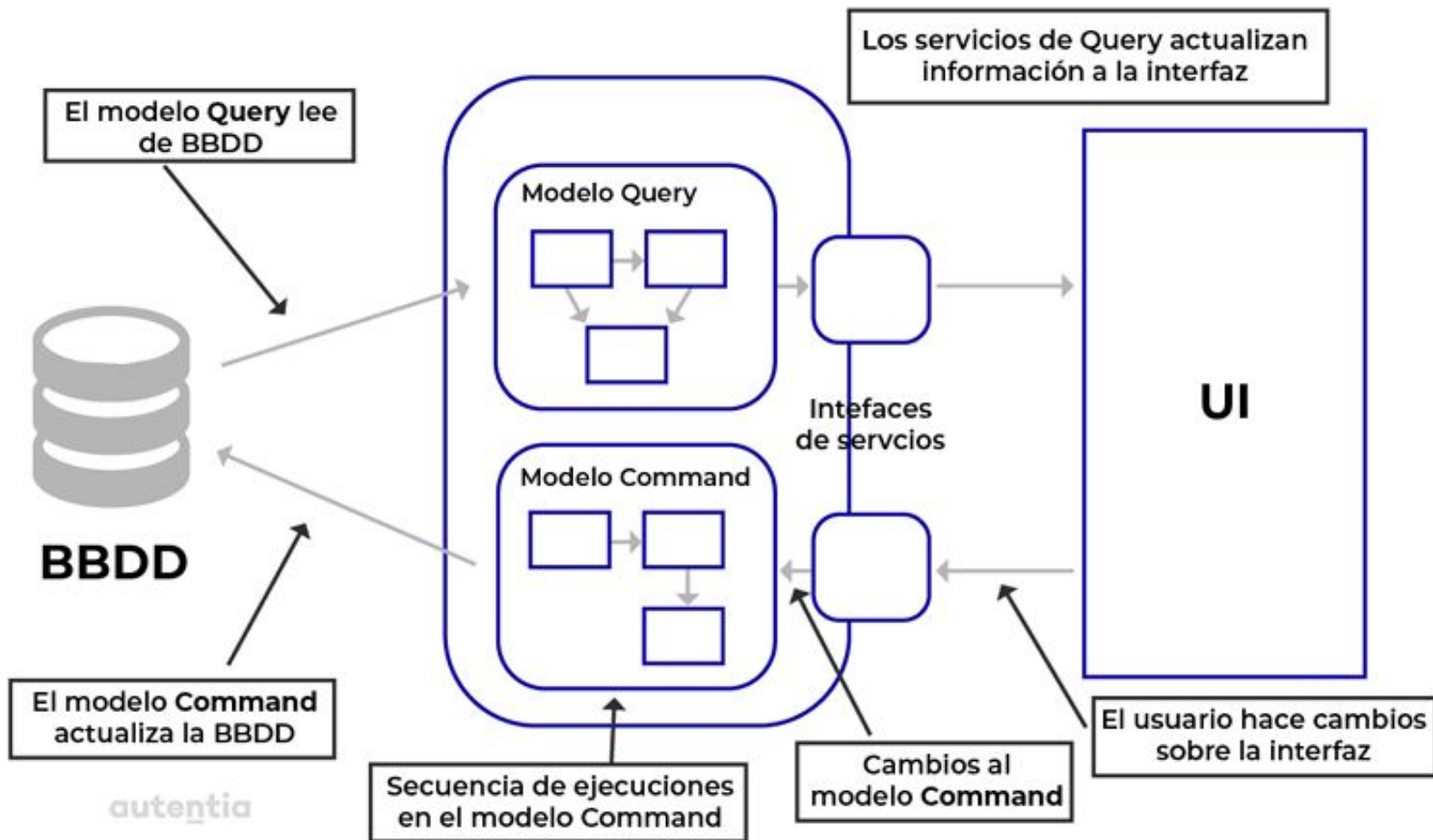
Por modelos separados, **comúnmente nos referimos a diferentes modelos de objetos**, probablemente ejecutándose en diferentes procesos lógicos, a veces incluso en hardware separado.

Es posible que los dos modelos, el de consultas y el de comandos, no sean modelos de objetos separados. Podría darse el caso de que los mismos objetos tengan interfaces diferentes para la parte de Consultas y la de Comandos.

- Como cualquier patrón, **CQRS es útil en algunos lugares** pero no en otros, por lo que no debe abordarse a menos que el beneficio valga la pena.
- **CQRS te permite separar la carga de las lecturas y escrituras**, lo que permite escalar cada una de forma independiente.



Fuente: <https://martinfowler.com/bliki/CQRS.html>





¿CÓMO ENCAJA CQRS CON OTROS PATRONES DE ARQUITECTURA?

CQRS encaja naturalmente con algunos otros patrones de arquitectura.

- A medida que nos alejamos de una única representación con la que interactuamos a través de CRUD, podemos pasar fácilmente a una interfaz de usuario basada en tareas.
- **CQRS encaja bien con los modelos de programación basados en eventos.** Es común ver el sistema CQRS dividido en servicios separados que se comunican con Event Collaboration. Esto permite que estos servicios aprovechen fácilmente el Abastecimiento de eventos.
- **Tener modelos separados plantea preguntas** acerca de cuán difícil es mantener esos modelos consistentes, lo que aumenta la probabilidad de usar una consistencia eventual.
- **Para muchos dominios**, se necesita gran parte de la lógica cuando se actualiza, por lo que puede tener sentido usar *EagerReadDerivation* para simplificar sus modelos del lado de consulta.
- **Si el modelo de escritura genera eventos para todas las actualizaciones**, puede estructurar los modelos de lectura como *EventPosters*, lo que les permite ser *MemoryImages* y evitar así muchas interacciones de la base de datos.
- **CQRS es adecuado para dominios complejos**, del tipo que también se beneficia del diseño impulsado por dominio.



¿CUÁNDO UTILIZAR CQRS?

Como cualquier patrón, **CQRS es útil en algunos lugares**, pero no en otros por lo que **no debe abordarse a menos que el beneficio valga la pena**.

En particular, **CQRS solo debería usarse en partes específicas de un sistema** (un BoundedContext en la jerga DDD) y no en el sistema en su conjunto. En esta forma de pensar, cada Bounded Context necesita sus propias decisiones sobre cómo debería modelarse.

Se han observado beneficios de uso en CQRS en dos direcciones:

Fuente: <https://martinfowler.com/bliki/CQRS.html>

- En primer lugar, **es posible que algunos dominios complejos sean más fáciles de abordar utilizando CQRS**.
- **CQRS te permite separar la carga de las lecturas y escrituras**, lo que te permite escalar cada una de forma independiente. Si tu aplicación ve una gran disparidad entre lecturas y escrituras, te va a resultar muy útil.
- **Si su dominio no es adecuado para CQRS**, pero tienes consultas exigentes que agregan problemas de complejidad o rendimiento, recuerda que puedes usar una base de datos de informes.



¿Qué es?

Es una **aplicación pequeña que ejecuta su propio proceso y se comunica mediante mecanismos ligeros** (normalmente una API de recursos HTTP). Cada aplicación se encarga de implementar una funcionalidad completa del negocio, es desplegado de forma independiente y puede estar programado en distintos lenguajes así como usar diferentes tecnologías de almacenamiento de datos.



CARACTERÍSTICAS PRINCIPALES

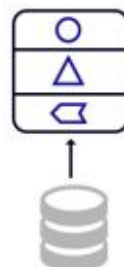
Hasta hace unos años, las aplicaciones grandes y complejas se implementaban como grandes monolitos muy difíciles de mantener y evolucionar. Una arquitectura basada en microservicios **consiste en implementar los distintos servicios de nuestra aplicación a través de servicios más pequeños e independientes entre sí.**

Estos servicios se caracterizan por:

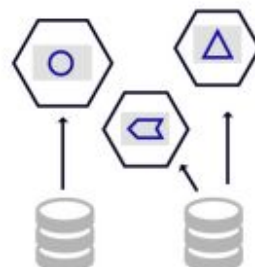
- Poco acoplamiento.
- Mantenibilidad.
- Totalmente independientes del resto de microservicios.
- Cada uno implementa una arista de la aplicación de negocio (p. ej. microservicio de gestión de usuarios).

La decisión de si utilizar o no este tipo de diseño a la hora de construir nuestro sistema, **se fundamenta básicamente en el nivel de complejidad que va a alcanzar.** Para una aplicación con una complejidad baja no se recomienda utilizar esta arquitectura.

MONOLITOS



MICROSERVICIOS



VENTAJAS DE LOS MICROSERVICIOS

- Escalabilidad más eficiente e independiente.
- Pruebas más concretas y específicas.
- Posibilidad del uso de distintas tecnologías e implementaciones.
- Desarrollos independientes y paralelos.
- Aumento de la tolerancia a fallos.
- Mejora de la mantenibilidad.
- Permite el despliegue independiente.



¿Qué es?

En una aplicación monolítica, la comunicación entre los distintos componentes y verticales se realiza a través de llamadas internas entre los componentes. En una **arquitectura basada en microservicios** es necesario un mecanismo de **comunicación entre ellos para permitir la propagación de información** o informar de cierto evento en el microservicio emisor que desencadena una acción en el microservicio receptor.



TIPOS DE COMUNICACIÓN

Al ser cada microservicio una unidad independiente, es necesario establecer un protocolo de comunicación entre ellos. Si clasificamos según el **tipo de comunicación** tenemos una comunicación **síncrona/asíncrona**:

- **Síncrona:** el microservicio emisor se queda esperando la respuesta del receptor (p.e. **HTTP o HTTPS**). Implica que los microservicios no sean muy complejos y tengan buen rendimiento.
- **Asíncrona:** el microservicio emisor continúa su ejecución tras informar al receptor (p.e. **Basada en eventos** con RabbitMQ o AMQP o HTTP). En este caso la respuesta del receptor no es necesaria para continuar con la ejecución del emisor.



TECNOLOGÍAS

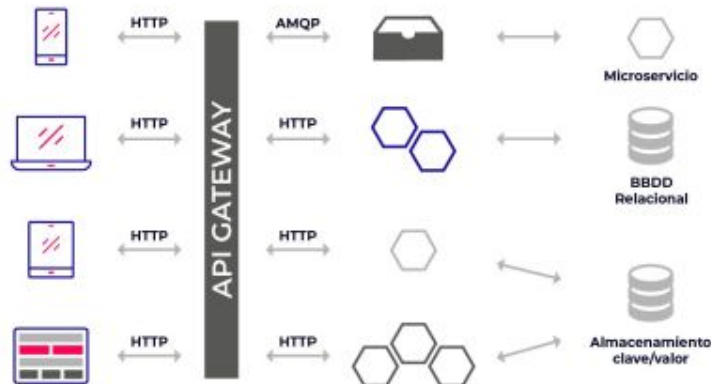
- API REST para comunicación a través de HTTP/HTTPS.
- RabbitMQ, Kafka, AMQP para comunicación basada en eventos/mensajes (uno-a-uno o uno-a-muchos).



NÚMERO DE RECEPTORES

En la comunicación **uno-a-uno** la comunicación se establece entre un emisor y un solo receptor. Esta comunicación puede ser de tipo **comando**, de manera que el receptor realice una acción concreta.

En la comunicación **uno-a-varios** el emisor informa a varios microservicios de ciertos eventos. La comunicación normalmente se implementa a través de un canal al que se suscriben todos los microservicios interesados y donde el receptor publica un mensaje o evento.

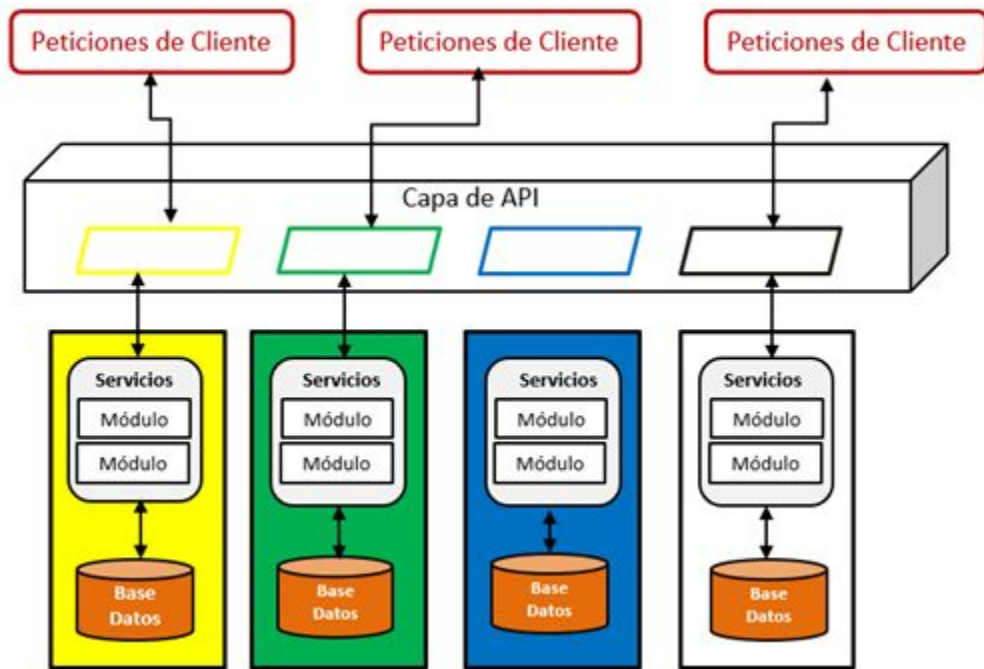


Microservicios - Ejemplo

Netflix: Pionera en la implementación de este patrón arquitectónico, cuenta con una plataforma basada Microservicios desde hace años.

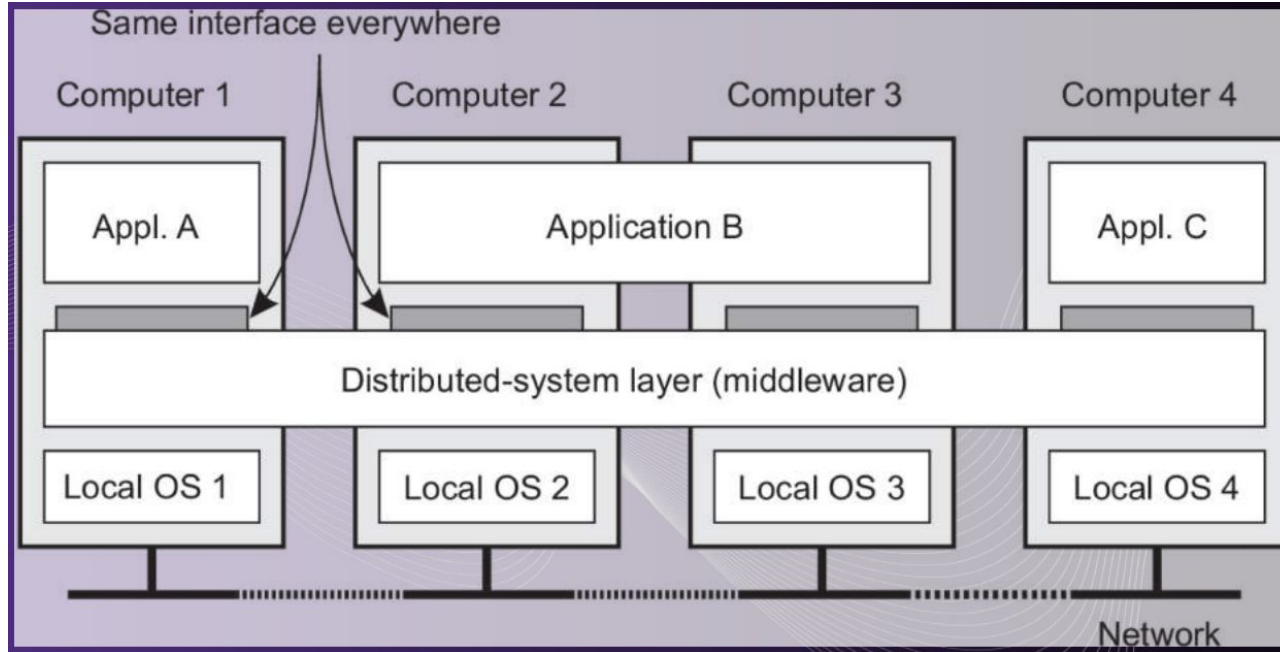
Según algunas publicaciones actualmente recibe una media de mil millones de llamadas a sus diferentes servicios y es capaz de adaptarse a más de 800 tipos de dispositivos mediante su API de streaming de vídeo, la cual realiza cinco solicitudes a diferentes servidores para no perder nunca la continuidad de la transmisión.

Componentes de software autónomos que se centran en una funcionalidad específica (como pagos, usuarios, streaming, etc.)



Sistemas distribuidos

Un sistema distribuido es un **conjunto de componentes de hardware o software ubicados en computadoras interconectadas a través de una red**, que trabajan juntos para lograr un objetivo común. Estos sistemas están diseñados para **funcionar como una única entidad coherente**, aunque cada componente individual puede ejecutarse en máquinas diferentes y estar geográficamente disperso.



El foco es
computación en
múltiples nodos.

P ejm. Blockchain

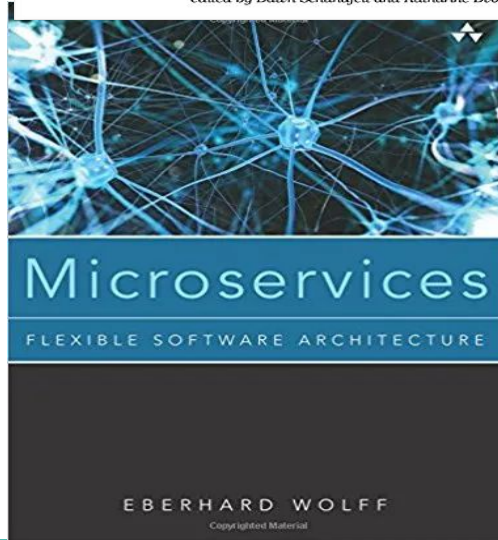
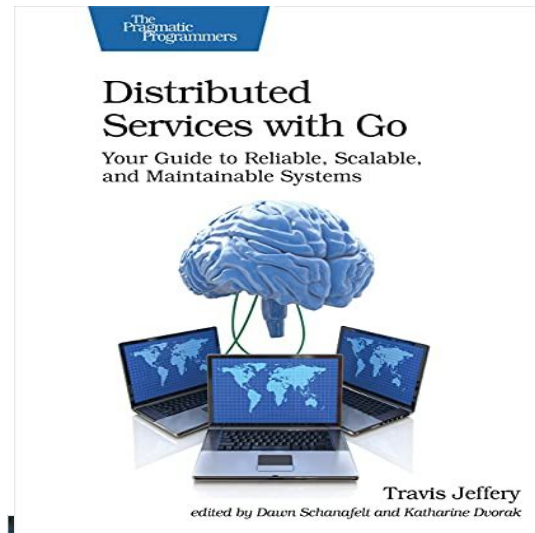
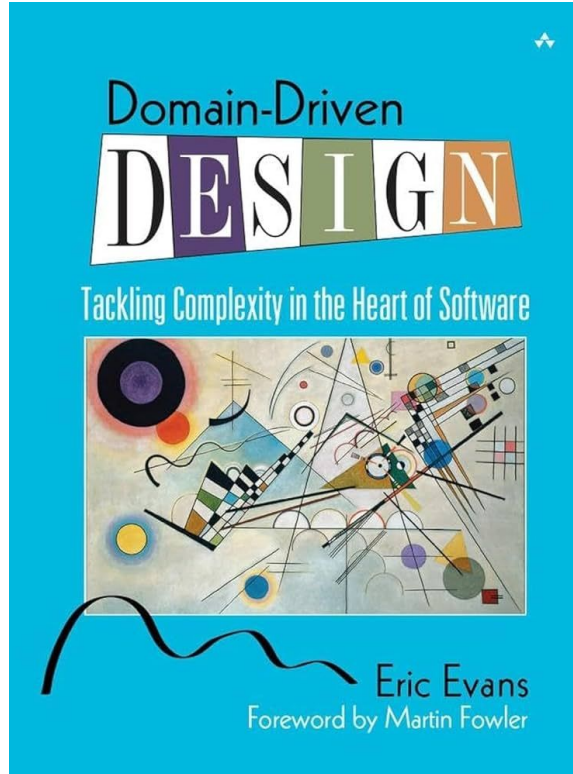
Sistemas distribuidos

Algunas características clave de los sistemas distribuidos incluyen:

- **Concurrencia:** Múltiples componentes del sistema pueden ejecutar tareas simultáneamente, lo que permite un procesamiento paralelo y una mayor capacidad de respuesta.
- **Modularidad y escalabilidad:** Los sistemas distribuidos pueden escalar horizontalmente agregando más nodos a la red, lo que les permite manejar grandes volúmenes de datos y aumentar su capacidad de procesamiento según sea necesario.
- **Tolerancia a fallos:** Los sistemas distribuidos están diseñados para ser robustos frente a fallos individuales de componentes, ya sea hardware o software. Esto se logra mediante técnicas como la redundancia, la replicación de datos y los mecanismos de recuperación.
- **Independencia de componentes:** Los nodos pueden tener hardware, software y sistemas operativos diferentes, pero trabajan coordinadamente.
- **Ausencia de reloj global:** No existe un tiempo global sincronizado entre nodos, lo que complica la coordinación temporal.

Los sistemas distribuidos se utilizan en una amplia variedad de aplicaciones, desde servicios en la nube y redes sociales hasta sistemas de gestión de bases de datos distribuidas y sistemas de procesamiento genómico e inteligencia artificial. La naturaleza distribuida de estos sistemas proporciona beneficios como la escalabilidad, la tolerancia a fallos y la redundancia, pero también presenta desafíos en términos de coordinación, consistencia de datos y seguridad.

Lecturas extra - caps. 6, 13, 18, 19 (Sommerville)



Tarea

Para cada enfoque arquitectónico visto hoy (**CQRS**, **Microservicios**, **Sistemas Distribuidos**) crear una tabla (similar a las del libro Sommerville que vimos para los patrones en capas, MVC, Cliente-Servidor, Pipe&Filter) incluyendo:

- Nombre
- Descripción
- Ejemplo - crear diagrama(s) para ilustrar, análogo a las figuras en el libro
- Cuándo se usa
- Ventajas
- Desventajas

Entrega por siveduc - **viernes 24/04**

Pausa 5 min.

Principios - Buenas Prácticas



¿Qué es?

Es una estrategia (entre otras) que nos permite mantener la información actualizada en sistemas distribuidos o basados en microservicios. A través de la consistencia eventual cuando uno de los microservicios modifica la información, el cambio se propaga a cada una de las partes que también la manejan.



EN QUÉ CONSISTE

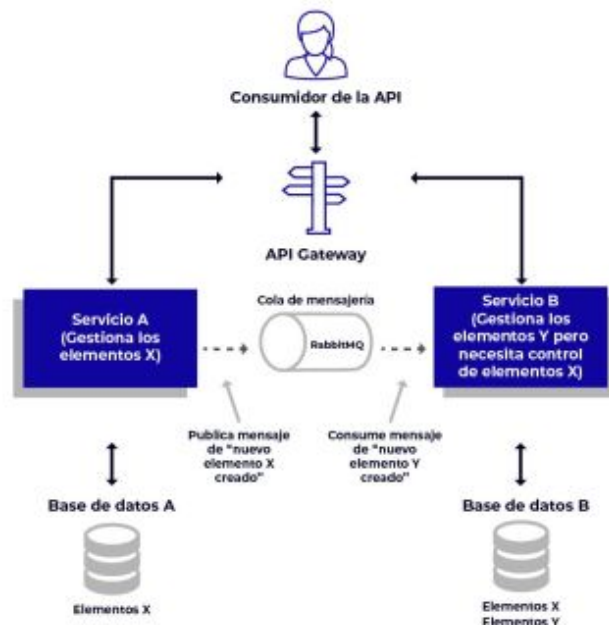
En un monolito, cada una de las funcionalidades existentes comparten la misma información debido a que la fuente de datos es la misma. La consistencia eventual nos permite en una arquitectura basada en microservicios, cada uno con su propia base de datos, posean la información actualizada y coherente.

La consistencia eventual puede implementarse a través de un modelo basado en eventos usando AMQP, RabbitMQ o Kafka.

Veamos el siguiente ejemplo:

- Dados los microservicios A y B, donde A es el microservicio que se encarga de la gestión de usuario y B es el encargado de la administración del sistema.
- Para que un usuario pueda realizar cualquier operación de administración, B debe comprobar que el usuario existe en el sistema.
- A lo registra en su base de datos y comunica a B que dé de alta en su base de datos al nuevo usuario.

Puede ocurrir que haya un momento en el tiempo en que la información no tenga consistencia. Ante estas situaciones hay que implementar algún mecanismo de recuperación para garantizar la consistencia de la información (p. ej. Dead Letter Queue, mantener un histórico de eventos enviados...).





¿Qué es?

El principio ACID es un estándar de las bases de datos relacionales que se deben cumplir para que se puedan realizar las transacciones en ellas. ACID es el acrónimo en inglés de **A**tomicity, **C**onsistency, **I**solation y **D**urability (Atomicidad, Consistencia, Aislamiento y Durabilidad).



EN DETALLE

Es muy probable que si trabajas o te has movido en torno a las bases de datos en algún momento, hayas oído hablar del término ACID. Este hace referencia a las propiedades que debe cumplir una transacción para que se considere confiable. El término fue ideado por Andreas Reuter y Theo Härde en 1983.

¿Qué entendemos por transacción? En base de datos una transacción consiste en una operación lógica. Por ejemplo, aumentar el IVA de los productos de nuestra base de datos es una única transacción pero puede conllevar acciones sobre numerosas tablas.

Veamos cada uno de los términos:

- **Atomicidad.** Una transacción puede estar compuesta por varias operaciones, si una operación falla todas fallan, por tanto la base de datos se mantiene inmutable.
- **Consistencia.** Asegura que cualquier transacción realizada llevará a la base de datos de un estado válido a otro válido. Por tanto, los datos deben respetar todas las reglas y restricciones definidas.
- **Aislamiento.** Asegura que la ejecución concurrente de varias transacciones deja a la base de datos igual que si estas se hubieran ejecutado de forma secuencial.
- **Durabilidad.** Indica que una vez confirmada una transacción los datos deben persistir en la base de datos

A

Atomicidad: las transacciones son todo o nada.

C

Consistencia: Sólo se guardan los datos que respetan las reglas definidas.

I

Aislamiento: Las transacciones no se afectan las unas a las otras.

D

Durabilidad: Los datos que se hayan escrito no se perderán.

Y.A.G.N.I - You Aren't Gonna Need It

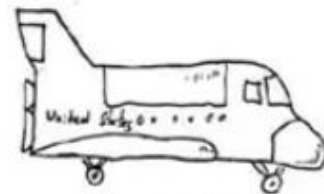


Software Cemetery

Y.A.G.N.I - You Aren't Gonna Need It



Good Bike



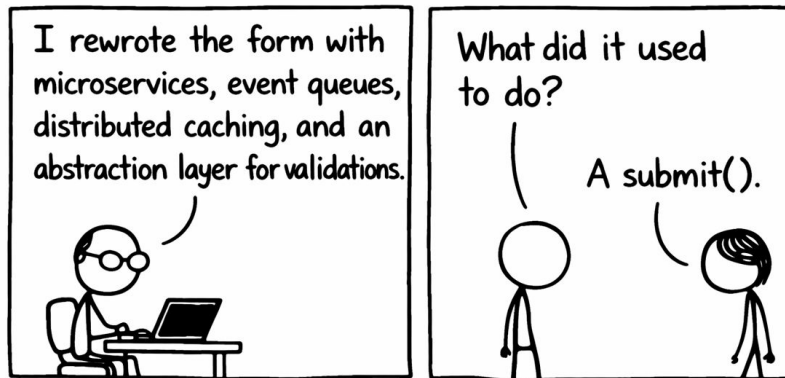
Bad Bike

Literalmente traducido por **no vas a necesitarlo**.

Una fuente común de errores es el código escrito “porque podríamos necesitarlo algún día” o en “creo que lo usaremos más adelante”. En general, el desarrollo de cualquier funcionalidad basada en la especulación es una bomba que tarde o temprano estallará.

No intentar hacer una mega-solución que solucione los problemas del mañana porque estos pueden cambiar, y, en lugar de esto, centrarse en únicamente lo necesario, que por otra parte es por lo que nos pagan.

K.I.S.S - Keep It Simple Stupid!



K.I.S.S.

The architecture was more complex than the problem.

K.I.S.S - Keep It Simple Stupid!

Aunque se ha traducido normalmente por **manténlo simple, idiota!**, el autor, Kelly Johnson lo ha traducido sin coma, lo que no insulta al ingeniero encargado sino que refuerza la idea de sencillez. Cualquier sistema va a funcionar mejor si se mantiene sencillo que si se vuelve complejo.

La sencillez tiene que ser una meta en el desarrollo y la complejidad innecesaria debe ser eliminada.

*"We should forget about small efficiencies, say about 97% of the time: **premature optimization is the root of all evil**. Yet we should not pass up our opportunities in that critical 3%."*

D.R.Y - Don't Repeat Yourself

~~D.I.E - Duplication Is Evil~~



D.R.Y - Don't Repeat Yourself

~~D.I.E - Duplication Is Evil~~

Se pueden agrupar estos dos conceptos porque su significado es el mismo, no te repitas y la duplicación es el mal.

Una fuente común de errores es el código repetido que hace cosas muy similares. Los errores se introducen en los sistemas como código repetido cuando se agrega una nueva funcionalidad y solo se actualiza en una de las dos partes de código similar.

Peor aún, si hay un error existente, es posible que solo se solucione en un lado, lo que hace que el error pueda volver a ocurrir debido a la corrección incompleta.

The boy scout rule

[Code](#)[Issues](#)[Pull Requests](#)[Actions](#)[Projects](#)[Watch](#)

1

[☆ 23](#)[5](#)

Commit: fix button color

2 hours ago



Author: **dev123** ▾

Archivos cambiados: 28

Tests agregados: 4

Función duplicada eliminada: 1

Variable tmp2_final_final renombrada: 1

Comentario: "Ya que estaba ahí."

[Browse files](#)[Copy SHA](#)

The boy scout rule

Los Boy Scouts tienen la regla de dejar el campamento más limpio de lo que se lo encontraron y en caso de ensuciarlo, se limpia y se deja lo mejor posible para la siguiente persona que venga.

Si se aplica esto al desarrollo de software, se puede decir que si vemos alguna parte del código que se pueda mejorar, independientemente de quién lo haya hecho, debemos hacerlo. El objetivo principal es mejorar la calidad del código y evitar su deterioro con el fin de ayudar al siguiente desarrollador (o a uno mismo dentro de un tiempo), a cambiar o desarrollar una nueva funcionalidad de una forma más sencilla.